

Catching Phishing from the Link Alone

A Critical Evaluation of URL-Based Phishing Detection

Gilad Segal

Data Science & Cyber — Dr. Uri Itai · URL / link detector (one half of a two-part project)

Code, data pipeline and reproducible results: github.com/segals/phishing-url-detector

Abstract

Phishing remains a leading initial-access vector, and a substantial fraction of it is carried entirely by a malicious link. This report presents a URL-only phishing detector and subjects it to a rigorous critical evaluation, rather than treating a high benchmark score as the end of the analysis. Trained on 235,795 real URLs (PhiUSIIL [1]), a gradient-boosted classifier over 40 hand-engineered lexical features reaches $F1 = 0.997$ in-distribution, matching the near-perfect scores reported in the literature. Using single-feature ablation, explainability (SHAP [7]) and a feature-group ablation, we show that this score is driven overwhelmingly by a dataset collection artifact — legitimate URLs in this corpus are 100% HTTPS versus 48.6% for phishing — rather than by a transferable phishing signal. An adversarial evaluation shows the detector is robust to character-level obfuscation (homoglyphs, typosquatting; recall ≥ 0.99) but is evaded completely (recall = 0.00) by hosting phishing content on ordinary HTTPS infrastructure with a clean, short URL. Most critically, evaluated without retraining on an independent 651,191-URL corpus [2], the detector collapses to $F1 = 0.666$, $AUC = 0.434$ — below 0.5, meaning its ranking is inverted rather than merely uninformative — while a distance-metric analysis (Kolmogorov–Smirnov, Wasserstein) confirms the collapse is driven by exactly the features the model relies on most. On a live feed of current phishing URLs [3], however, recall remains 1.00. We conclude that URL structure alone is not a generalisable phishing signal, and argue for its use as one independent channel within a fused, multi-signal detector.

1. Introduction

Phishing attacks rely on a victim following a malicious link, so the URL itself is an attractive detection surface: it can be scored in milliseconds, requires no page fetch, preserves user privacy, and can be evaluated before the user clicks. A large body of published work reports near-perfect ($F1 > 0.99$) classification accuracy on public URL datasets, which could suggest the problem is essentially solved. This report interrogates that claim.

The project is deliberately structured as a critical evaluation rather than a model-accuracy exercise: we reproduce the strong published result, then systematically ask whether it reflects genuine phishing-specific signal or an artifact of how the benchmark dataset was constructed. The investigation is organised around four research questions.

RQ1 (signal). How much of the reported near-perfect score survives once dataset-specific leakage and collection bias are accounted for? **RQ2 (robustness).** How does the detector respond to an adversary who deliberately reshapes a phishing URL's surface features? **RQ3 (generalisation).** Does the in-distribution score transfer to an independent dataset and to live, current phishing traffic? **RQ4 (explainability).** Which features actually drive the decision, and does that explain the robustness and generalisation results?

This module is one of two independent, complementary detectors built for the course project: this report covers the **URL/link** channel, while a companion module covers the **email content** channel. The two were designed and evaluated independently; fusing them is identified as future work in Section 7. The contribution of this half is not a new model architecture but an honest, reproducible audit of a standard one.

2. Background and Related Work

Lexical and host-based URL features for phishing detection were established by Ma *et al.* [4], who showed that structural properties of a URL — its length, host composition, presence of an IP literal, and token content — carry substantial discriminative signal without inspecting page content. The feature families used in Section 4.1 follow that tradition. Deep, character-level models such as URLNet [5] instead learn a representation directly from the

URL string, trading interpretability for potential accuracy gains. We deliberately favour interpretable, hand-engineered features: because the central finding of this report is that a near-perfect score is attributable to a specific, nameable artifact, a model whose internal representation is opaque would have obscured precisely the evidence we set out to find.

On the adversarial side, Boucher *et al.* [6] document imperceptible-perturbation attacks against text-processing and web-facing systems, including homoglyph substitution — visually identical characters drawn from other Unicode scripts. This motivates both the homoglyph attack and the canonicalisation defence evaluated in Section 5.7. For explainability we use SHAP [7], a game-theoretic feature-attribution method that assigns each feature a principled share of a prediction and, unlike a tree model's built-in importance, is defined per-prediction and sums exactly to the model output. Our central methodological concern — that reported phishing-detection accuracy may not transfer across independently collected corpora — is treated here as a hypothesis to be tested empirically (Sections 5.9–5.10) rather than assumed.

3. Data

3.1 Primary corpus

The primary dataset is **PhiUSIIL** [1] (UCI Machine Learning Repository, id 967), comprising 235,795 real URLs with a raw URL string, a binary label, and 53 pre-computed descriptive features. It is among the largest and most recent public URL datasets and is a common benchmark, which motivates its use here as the primary object of study. Table 1 summarises the corpus after cleaning; Figure 1 shows the class balance and two representative feature distributions.

dataset	urls	phishing	legitimate	phishing_rate
PhiUSIIL	235370	100520	134850	0.427

Table 1. Composition of the primary PhiUSIIL corpus after cleaning.

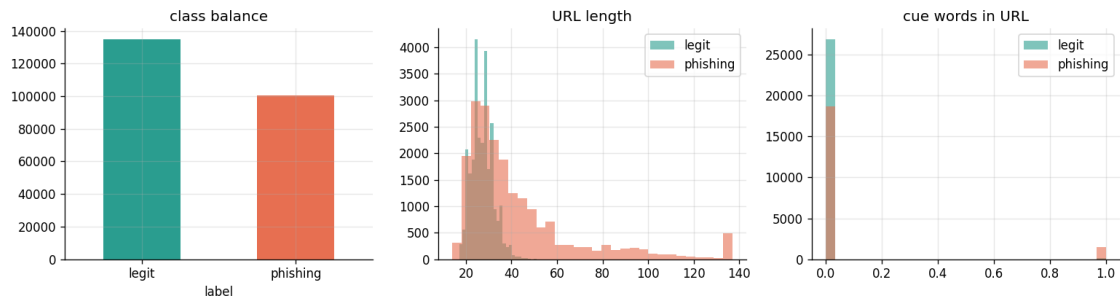


Figure 1. Exploratory view of the corpus: class balance (left), URL-length distribution by class (centre) and cue-word count by class (right). The classes separate almost perfectly on trivial structural quantities — an early indication that the task is easier than the real-world problem it represents.

3.2 Secondary corpora

Two independent sources support the generalisation analysis in Section 5.9. The Kaggle *Malicious URLs* dataset [2] contains 651,191 URLs labelled benign, phishing, defacement or malware; we use the benign/phishing subset, balanced to 40,000 URLs per class. It was collected independently of PhiUSIIL and is itself mixed-class, which makes it a fair generalisation test rather than a comparison against an unlabelled list. In addition, a live snapshot of the **OpenPhish** community feed [3] (300 currently active phishing URLs) provides a check against present-day, real-world attacks rather than an archived benchmark.

3.3 Data preparation

Data preparation was kept deliberately minimal, so that the results reported below are properties of the source data rather than of our own preprocessing. Labels were standardised to the convention 1 = phishing, 0 = legitimate (PhiUSIIL natively encodes the opposite polarity). Rows with a missing URL or label were dropped and

surrounding whitespace stripped. Rows were then **de-duplicated on the exact URL string before splitting** (235,795 → 235,370 rows), so that no URL can appear in both the training and test partitions — a common and easily overlooked source of leakage. No class rebalancing, outlier removal, or imputation was performed, and the natural class ratio of 42.7% phishing is retained throughout. Feature standardisation is applied only inside the logistic-regression pipeline, where it is required; the tree ensembles are scale-invariant and consume raw feature values.

The data were split 60/20/20 into training, validation and test partitions using a fixed random seed (42) with stratification on the label, so all three partitions carry the same class ratio. All model selection and tuning decisions were made on the validation partition; the test partition was evaluated once. For reproducibility the cleaned dataset is hashed and logged: `rows=235370, phishing_rate=0.4271, sha256=b39e8c2251df...` (full value in `results/tables/data_hash.txt`).

4. Methodology

4.1 Feature engineering

A URL is a short, structured string rather than free prose, so instead of learning a text representation (as a bag-of-words or TF-IDF pipeline would for email text) we compute interpretable numeric features directly from the string. All 40 features are derived from the URL alone and require no network access, keeping the detector fast, private and reproducible. They fall into six families, summarised in Table 2.

family	what it captures	examples
Lexical / length	the size and punctuation profile of the string	<code>url_len</code> , <code>n_dots</code> , <code>n_slash</code> , <code>n_digit</code> , <code>digit_ratio</code>
Host / domain	structure of the hostname itself	<code>n_subdomain</code> , <code>is_ip</code> , <code>has_port</code> , <code>core_len</code>
TLD	the suffix and its abuse association	<code>tld_len</code> , <code>suspicious_tld</code> (.tk, .xyz, ...)
Entropy	randomness of the string (generated domains)	<code>url_entropy</code> , <code>host_entropy</code> , <code>core_entropy</code>
Scheme / encoding	transport and character-encoding tricks	<code>is_https</code> , <code>has_punycode</code> , <code>has_hex_encoding</code> , <code>has_at_symbol</code>
Brand / cue	impersonation and social-engineering markers	<code>brand_min_dist</code> , <code>brand_lookalike</code> , <code>n_cue_words</code> , <code>is_shortener</code>

Table 2. The six feature families (40 features total), all computed from the URL string alone.

Two families rest on explicit theory. **Entropy** features use the Shannon entropy of a string, $H(s) = -\sum_i p_i \log p_i$, computed over its character distribution. A predictable, repetitive string such as *google* has low entropy, whereas an algorithmically generated domain such as *x7qz9wvt* approaches random noise and scores high; this is a standard signal for domain-generation algorithms. **Brand** features use the Levenshtein edit distance — the minimum number of single-character insertions, deletions or substitutions needed to transform one string into another, computed by the standard dynamic-programming recurrence — between each host label and a list of eighteen frequently impersonated brands. A label within edit distance 1–2 of a real brand (for example *paypa1* versus *paypal*) is flagged as a probable typosquat.

4.2 Models

Three classifiers were compared and selected on validation F1: **logistic regression** (a linear, directly interpretable baseline, with standardised features), **random forest** (an ensemble of decision trees trained independently on bootstrap samples and averaged), and **histogram-based gradient boosting** (trees fitted sequentially, each correcting the residual error of its predecessors, with features bucketed into histograms for speed on large data). The two tree ensembles represent the two standard ensembling strategies — variance reduction by averaging, and bias reduction by sequential correction — and neither requires feature scaling.

4.3 Evaluation protocol

Because the majority class accounts for 57.3% of the data, accuracy alone is not reported as a primary metric: a majority-class classifier attains 57.3% accuracy while catching zero phishing URLs, the classic *accuracy paradox* (Section 5.1). We therefore report precision, recall, F1, F2, the Matthews Correlation Coefficient (MCC) and AUC-ROC. **F2** weights recall more heavily than precision, reflecting the operational asymmetry that a missed phishing URL (false negative) is more costly than a false alarm. **MCC** is computed from all four confusion-matrix cells and ranges from -1 to $+1$, with 0 indicating chance performance; unlike accuracy or F1 it cannot be inflated by exploiting the majority class, which makes it the most trustworthy single figure under imbalance. **AUC-ROC** is the probability that a randomly drawn phishing URL is scored above a randomly drawn legitimate one, so 0.5 indicates uninformative ranking — and, importantly for Section 5.9, a value below 0.5 indicates a systematically inverted ranking rather than an absence of signal.

Every headline metric is reported with a **bootstrap 95% confidence interval**: the test set is resampled with replacement 1,000 times, the metric recomputed on each resample, and the 2.5th and 97.5th percentiles reported. This quantifies how much a metric would vary on comparable data using only the data already collected, and prevents over-reading small differences between models. Paired model comparisons additionally use **McNemar's test** on the discordant predictions of two models over the same test set, so that no comparative claim rests on a point estimate alone.

4.4 Threat model and adversarial evaluation

We model an attacker with black-box access who may freely choose the surface form of a phishing URL but must keep it functional — it must still resolve to attacker-controlled content. Four perturbation families are evaluated: *homoglyph substitution* (Latin characters replaced with visually identical Cyrillic or Greek confusables [6]); *typosquatting* (a single character duplicated or inserted in the registered domain); *scheme upgrade* (serving the identical page over HTTPS rather than HTTP); and *home-page mimicry* (a short, clean, path-free HTTPS domain, approximating the increasingly common abuse of legitimate cloud-hosting infrastructure). Two defences are evaluated: *canonicalisation*, which folds confusable characters back to their ASCII skeleton and decodes punycode before scoring, and *de-biasing*, which retrains the model with the HTTPS feature removed in order to test whether the artifact is concentrated in a single feature or distributed across correlated ones.

4.5 Generalisation and diagnostic tools

Cross-dataset generalisation is measured by training exclusively on PhiUSIIL and evaluating, with no retraining or recalibration, on the independent Kaggle corpus and the live OpenPhish feed. To diagnose *why* a gap occurs, we compare the per-feature distributions of the two datasets using the **Kolmogorov–Smirnov statistic** (the maximum vertical distance between two empirical cumulative distribution functions) and the **Wasserstein**, or earth-mover's, distance (the minimum probability mass that must be moved, weighted by the distance moved, to transform one distribution into the other), normalised by the in-distribution standard deviation so that features on different scales remain comparable. We additionally report two label-free anomaly detectors trained only on legitimate URLs as a weaker-assumption point of comparison, and assess probability calibration using the Brier score and a reliability curve.

5. Results

5.1 Trivial baselines and the accuracy paradox

Table 3 reports three baselines that require no learning. A majority-class predictor attains 57.3% accuracy with zero recall and an MCC of exactly 0 — a model that is useless for security but would appear respectable if judged on accuracy. More revealing is the second baseline: a single hand-written rule that flags any non-HTTPS URL as phishing already attains $F1 = 0.68$ and $MCC = 0.62$. That a one-line rule captures so much of the task foreshadows the artifact analysed in Section 5.2. By contrast the cue-word rule — arguably the most intuitively 'phishing-like' heuristic — performs poorly ($F1 = 0.13$).

baseline	accuracy	recall	f1	mcc
majority class (predict legit)	0.573	0.0	0.0	0.0
rule: not https -> phishing	0.793	0.516	0.68	0.616
rule: has cue word -> phishing	0.602	0.072	0.134	0.188

Table 3. Trivial baselines. A single HTTPS check alone reaches F1 0.68, while the intuitive cue-word rule reaches only 0.13.

5.2 Decomposing the near-perfect score

Training on all 50 numeric features provided with the dataset reproduces the near-perfect result reported in the literature ($F1 = 1.000$). Decomposing that score is the central diagnostic of this report (Table 4). A single provided feature, *URLSimilarityIndex*, attains $F1 = 0.995$ on its own. Inspection of the raw data explains why: this feature takes the value 100 for **100% of legitimate URLs**, while phishing URLs average 49.6 — it is effectively a re-encoding of the label rather than an independent predictor, and it could not be computed at deployment time for a genuinely unseen URL. It is therefore treated here as leakage, measured once as evidence and **excluded from every subsequent experiment**; the detector evaluated throughout the remainder of this report uses only the 40 features described in Section 4.1.

representation	f1	auc	f1_95ci
all 50 provided features (RF)	1.0	1.0	[1.000, 1.000]
single provided feature: <i>URLSimilarityIndex</i>	0.995	0.996	nan
single provided feature: <i>URLCharProb</i>	0.62	0.766	nan
single provided feature: <i>TLDLegitimateProb</i>	0.0	0.607	nan
honest raw-URL lexical (40, RF)	0.997	0.999	[0.996, 0.997]

Table 4. Decomposing the near-perfect score: all provided features, individual provided features, and our own leakage-free lexical features. Bootstrap 95% CIs shown where a full model was fitted.

The more interesting result is the last row: our own leakage-free features still attain $F1 = 0.997$. This is not explained by leakage, but by a second and deeper problem. As Table 5 shows, every legitimate URL in this corpus uses HTTPS, whereas fewer than half of the phishing URLs do. This 51-point gap is a property of how the two classes were collected — legitimate URLs appear to be drawn from clean, canonical home-pages and phishing URLs from messy, deep links — not a property of phishing itself. In the contemporary web the association is in fact much weaker, since the large majority of phishing sites now serve over HTTPS.

class	https_rate
legitimate	1.0
phishing	0.486

Table 5. HTTPS usage by class in PhiUSIIL: the dominant collection artifact.

5.3 Distribution shape, correlation, and non-parametric testing

URL-derived features are strongly right-skewed and heavy-tailed. Table 6 reports skewness and excess kurtosis for six representative features: URL length has a skewness of 54.7 and an excess kurtosis of 5,812, indicating a small number of extremely long URLs far from the bulk of the distribution. Under such distributions the mean and Pearson correlation are unreliable summaries, which motivates the use of rank-based statistics and medians throughout.

feature	skewness	excess_kurtosis
url_len	54.65	5811.54
host_len	2.54	10.79
digit_ratio	3.26	12.33
host_entropy	-0.1	0.64
n_cue_words	7.29	66.27
n_subdomain	1.82	7.93

Table 6. Distribution shape of representative features. Large positive skewness and excess kurtosis indicate heavy right tails.

Table 7 compares Pearson (linear) and Spearman (monotone) correlation between each feature and the label. The two diverge substantially for several features — path length has $r = 0.17$ but $\rho = 0.67$ — confirming strong monotone but non-linear relationships that a linear coefficient alone would badly understate. The single strongest linear relationship is *is_https* ($r = \rho = -0.61$), consistent with Section 5.2. Figure 2 shows the Spearman correlation structure among the leading features, revealing that they are not independent: the path- and length-related features form a strongly inter-correlated block, which becomes important when interpreting the ablation in Section 5.6.

feature	pearson	spearman
n_slash	0.478	0.671
n_path_segments	0.497	0.671
path_len	0.168	0.669
is_https	-0.613	-0.613
n_digit	0.164	0.542
digit_ratio	0.433	0.538
url_len	0.222	0.38
n_hyphen	0.188	0.368
url_entropy	0.395	0.358
host_hyphen	0.305	0.318

Table 7. Feature-label correlation, Pearson versus Spearman (top 10 by |Spearman|).

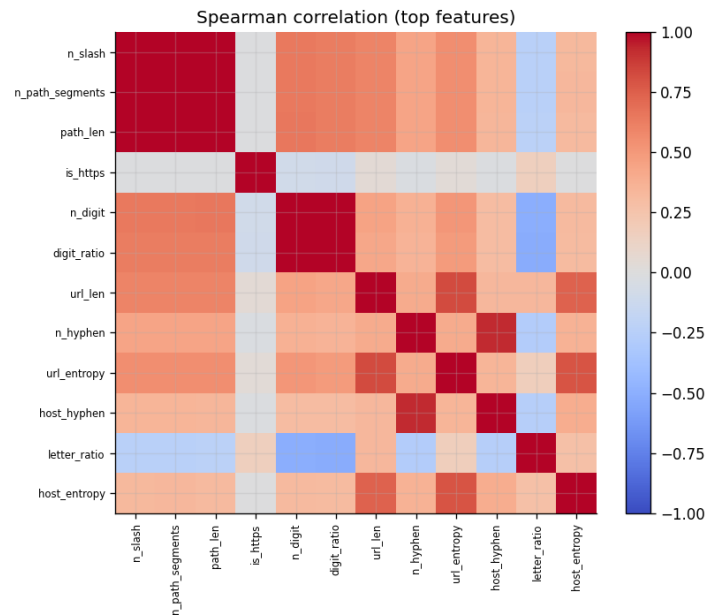


Figure 2. Spearman correlation among the leading features. Length- and path-related features form a correlated block, so they encode substantially overlapping information.

Finally, because the features are non-normal, class differences are tested with the non-parametric Mann–Whitney U test rather than a t-test (Table 8). Every feature tested differs significantly between classes, with p-values numerically indistinguishable from zero at machine precision — a consequence of the very large sample size as much as of effect size. The median values are more informative than the p-values: phishing URLs are longer (34 versus 27 characters) and marginally higher-entropy, while *is_https* shows a complete median separation (0 for phishing, 1 for legitimate), again isolating the artifact.

feature	mannwhitney_p	median_phishing	median_legit
url_len	0.0	34.0	27.0
host_entropy	0.0	3.625	3.424
digit_ratio	0.0	0.0	0.0
n_cue_words	0.0	0.0	0.0
is_https	0.0	0.0	1.0

Table 8. Mann–Whitney U tests of class differences, with class medians.

5.4 Model comparison

Table 9 compares the three candidate classifiers on the 40-feature representation. Random forest and histogram gradient boosting are statistically indistinguishable (McNemar $p = 1.00$, and their bootstrap confidence intervals coincide), while both significantly outperform logistic regression ($p = 7.7 \times 10^{-15}$) — a difference that is statistically real but practically negligible, at roughly three F1 thousandths. The confidence intervals are extremely tight (width ≈ 0.001), which matters for the argument of this report: the near-perfect score is stable and reproducible, not an artifact of a fortunate split. The tree ensemble is used as the reference detector for all subsequent experiments.

model	f1	f1_95ci	recall	precision	mcc	auc	mcnemar_vs
logreg	0.994	[0.993, 0.995]	0.99	0.999	0.99	0.998	$p=7.7e-15$
random_forest	0.997	[0.996, 0.997]	0.994	0.999	0.994	0.999	(best)
hist_gbm	0.997	[0.996, 0.997]	0.994	0.999	0.994	0.999	$p=1.0e+00$

Table 9. Model comparison with bootstrap 95% confidence intervals and McNemar tests against the best model.

5.5 Explainability

SHAP attribution [7] (Table 10, Figure 3) identifies *is_https* as the dominant feature, with a mean absolute SHAP value roughly three times that of the next-ranked feature. The remaining top features — path segments, path length, slash count — are structural descriptors of 'how deep' a URL is. The explanation therefore converges independently on the same conclusion as Section 5.2: the model is largely an HTTPS-and-path-depth detector. Figure 4 provides an independent cross-check using the random forest's own impurity-based importance, which ranks the same feature set — the agreement between two different attribution methods strengthens the finding.

feature	mean_abs_shap
is_https	0.205
n_path_segments	0.07
path_len	0.062
n_slash	0.05
n_digit	0.025
url_len	0.022
core_len	0.021
digit_ratio	0.02

Table 10. Global SHAP feature importance (top 8 of 40).

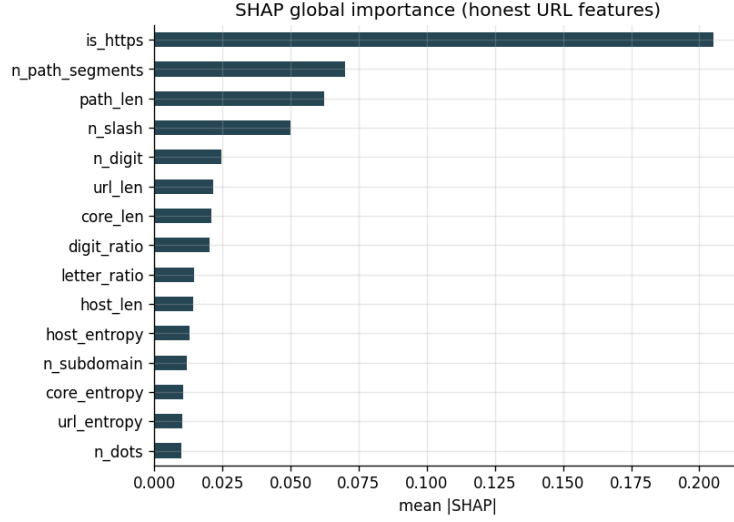


Figure 3. SHAP global feature importance. *is_https* dominates the model’s decisions by roughly a factor of three.

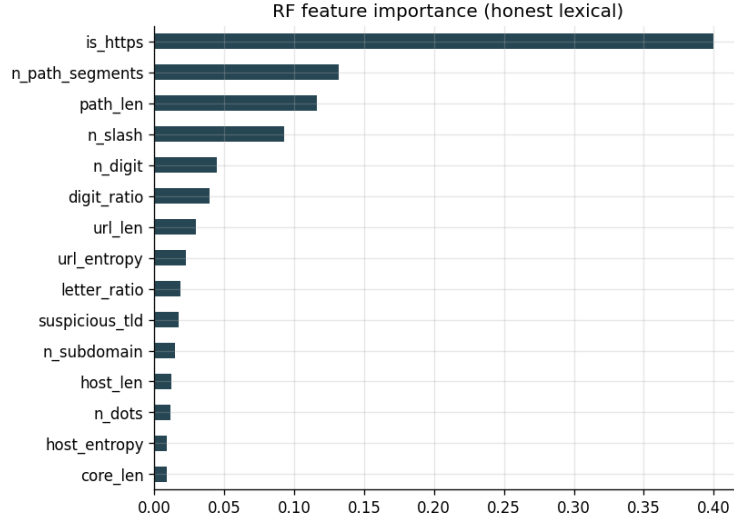


Figure 4. Independent cross-check: the random forest’s built-in impurity-based feature importance, which agrees with the SHAP ranking.

5.6 Feature-group ablation

Grouping the 40 features into the six families of Table 2 and training on each family in isolation (Table 11) shows that the artifact is not confined to a single feature. The *length/counts* family — eight features, none of them *is_https* — reaches $F1 = 0.993$ alone, nearly matching the full model, while the *scheme/encoding* family containing *is_https* reaches only 0.696 in isolation. Meanwhile the *brand/cue* family, which encodes the features most directly associated with phishing intent in prior work [4], is the weakest of all ($F1 = 0.273$). The interpretation, consistent with the correlation structure in Figure 2, is that the corpus encodes a broad ‘clean home-page versus deep messy link’ distinction across many correlated structural proxies. Any one of them can substitute for the others — which anticipates the limited effect of the de-biasing defence in the next section.

feature_group	n_features	f1	auc
length/counts	8	0.993	0.998
host/domain	5	0.687	0.823
tld	2	0.266	0.574
entropy	3	0.806	0.928
scheme/encoding	4	0.696	0.767
brand/cue	5	0.273	0.582

Table 11. Feature-group ablation. Length and count features alone nearly reproduce the full model, while brand and cue features contribute least.

5.7 Adversarial robustness

Table 12 and Figure 5 report recall on phishing URLs under each attack and defence. Character-level attacks leave recall essentially unchanged (homoglyph 0.992, typosquat 0.993 against a clean baseline of 0.994): a detector built on structural quantities is largely insensitive to which specific characters appear, in marked contrast to lexical text classifiers, which such attacks are known to defeat [6]. This is a genuine, if incidental, robustness property, and it is complementary to the content-based detector built in the companion module.

Along the axis on which the model is biased, however, it is highly vulnerable. A scheme upgrade alone — changing nothing but HTTP to HTTPS — reduces recall from 0.994 to 0.672, meaning roughly one third of phishing URLs cross the decision boundary as a result of a single free change. Home-page mimicry defeats the detector entirely (recall = 0.000): once a phishing URL is short, path-free and served over HTTPS, it matches every structural expectation the model has learned for legitimate traffic. Canonicalisation, as expected, has no effect on either attack, since there is no confusable character to fold; it is effective only against the homoglyph family it was designed for. Removing *is_https* and retraining recovers only marginally (0.672 → 0.704), confirming the ablation finding that the bias is diffuse. No defence available within a URL-only feature space closes this gap.

attack	recall_attacked	recall_+normalize	recall_+debias(no https)
(clean)	0.994	0.994	0.994
homoglyph	0.992	0.994	0.994
typosquat	0.993	0.993	0.994
https_upgrade	0.672	0.672	0.704
homepage_mimicry	0.0	0.0	0.0

Table 12. Recall on phishing URLs under each attack, with and without each defence.

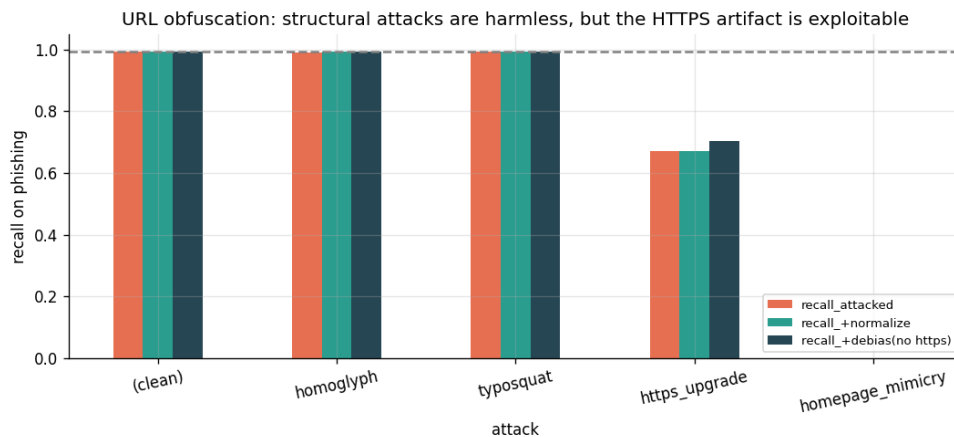


Figure 5. Recall under each attack. Structural (character-level) attacks are ineffective; attacks that exploit the HTTPS and path-depth artifact are highly effective.

5.8 Unsupervised detection and calibration

As a weaker-assumption comparison, two unsupervised anomaly detectors were trained exclusively on legitimate URLs, with no phishing labels at all (Table 13). Isolation Forest, which isolates anomalous points using random splits, attains AUC = 0.828; Local Outlier Factor, which compares a point's local density to that of its neighbours, attains AUC = 0.944. Both are creditable without any labelled attack data, though below the supervised model — and both are expected to depend on the same structural artifacts, since they measure structural unusualness rather than maliciousness.

unsupervised_detector	auc
IsolationForest	0.828
LocalOutlierFactor	0.944

Table 13. Unsupervised anomaly detectors trained only on legitimate URLs.

Probability calibration was assessed with the Brier score (the mean squared error between predicted probability and outcome) and a reliability curve, which plots observed frequency against predicted probability. In-distribution the detector is well calibrated overall (Brier = 0.0025), though Table 14 and Figure 6 show the expected instability in the sparsely populated middle of the probability range: the model is confidently correct at both extremes, where almost all of its predictions fall, and noisier in between. Section 5.9 shows that this calibration does not survive a change of dataset.

mean_predicted	observed_freq
0.003	0.003
0.141	0.293
0.234	0.273
0.34	0.5
0.449	0.25
0.56	0.5
0.634	0.556
0.745	0.364
0.85	0.958
1.0	1.0

Table 14. Reliability table: observed phishing frequency per predicted-probability bin.

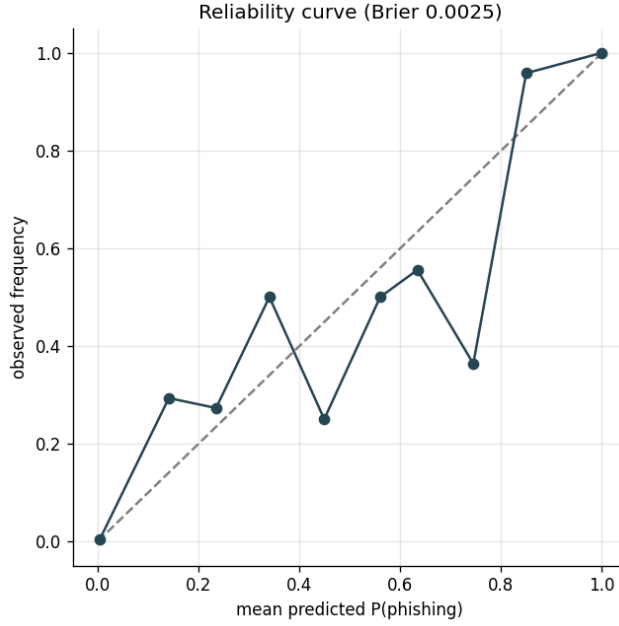


Figure 6. Reliability curve. Points on the diagonal indicate well-calibrated probabilities; the bulk of predictions lie at the two extremes.

5.9 Cross-dataset generalisation

The central generalisation result is reported in Table 15 and Figure 7. The in-distribution detector ($F1 = 0.997$, $AUC = 0.999$) collapses on the independent Kaggle corpus to $F1 = 0.666$, $AUC = 0.434$. Figure 7 makes the mechanism directly visible: in-distribution, the predicted-probability distributions of the two classes are cleanly separated, with legitimate URLs near 0 and phishing URLs near 1; on the independent corpus **both** classes collapse into the same region near 1.0 (mean predicted probability 1.000 for benign URLs and 0.999 for phishing URLs; 100.0% of benign and 99.9% of phishing URLs are scored above 0.9). The model has not merely become less confident — it no longer separates the classes at all.

An AUC below 0.5 reflects this precisely: within that saturated band, benign URLs are on average ranked marginally above phishing URLs, so the ranking is not merely uninformative but systematically inverted. This explains why re-thresholding recovers almost nothing ($F1\ 0.666 \rightarrow 0.667$ at the cost-optimal cut-off): re-thresholding can repair a model whose ranking is intact but whose operating point is mis-set, not one whose outputs no longer discriminate. Removing *is_https* does not help either — the de-biased variant is worse ($AUC = 0.168$) — indicating that the transferability failure is not attributable to that single feature. Notably, recall on the **live OpenPhish** feed remains 1.00: present-day phishing URLs are still structurally irregular enough to be caught. The correct reading is that the detector remains useful against today's attacks while being demonstrably non-transferable, and that its high recall on the independent corpus is accompanied by an unacceptable false-positive rate rather than genuine discrimination.

setting	f1	recall_phishing	auc	threshold
in-distribution (PhiUSIIL test)	0.997	0.994	0.999	0.5
cross-dataset (Kaggle, full model)	0.666	0.999	0.434	0.5
cross-dataset (Kaggle, no is_https)	0.666	0.998	0.168	0.5
cross-dataset (Kaggle, re-thresholded)	0.667	1.0	0.434	0.01
live OpenPhish (recall only)	nan	1.0	nan	0.5

Table 15. Cross-dataset generalisation: in-distribution performance versus an independent 651k-URL corpus and a live phishing feed.

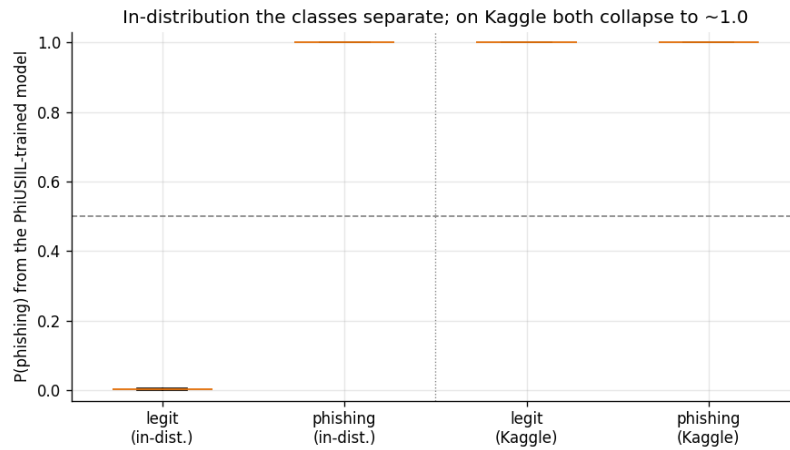


Figure 7. Predicted-probability distributions by true class. In-distribution the classes are cleanly separated; on the independent corpus both collapse into the same saturated region.

5.10 Diagnosing the collapse

To identify why generalisation fails, Table 16 ranks features by the distributional distance between their PhiUSIIL and Kaggle distributions. The three most-shifted features by the Kolmogorov–Smirnov statistic — *is_https* (0.741), *path_len* (0.694) and *n_path_segments* (0.627) — are precisely the features SHAP identified as the model’s strongest decision drivers in Section 5.5. The normalised Wasserstein distances tell the same story. The detector’s accuracy is therefore built on the least transferable part of the feature space, which is a mechanistic and quantitative explanation of the collapse in Section 5.9 rather than a restatement of it. Two independent diagnostic tools — feature attribution and distributional distance — converge on the same small set of features, which is strong evidence for the causal account offered here.

feature	ks_statistic	wasserstein_norm
is_https	0.741	1.789
path_len	0.694	0.931
n_path_segments	0.627	1.915
n_subdomain	0.435	0.968
url_len	0.343	0.434
n_slash	0.309	1.06
letter_ratio	0.294	0.446
n_dots	0.28	0.559

Table 16. Distributional shift between PhiUSIIL and the independent corpus, by Kolmogorov–Smirnov statistic and normalised Wasserstein distance (top 8 features).

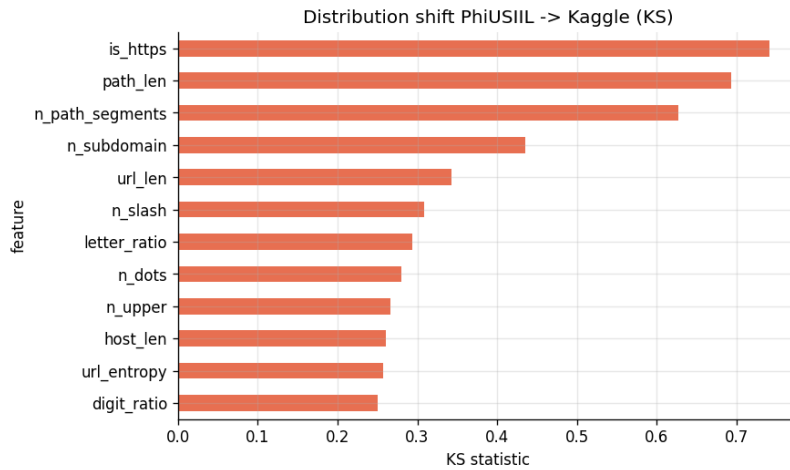


Figure 8. Kolmogorov–Smirnov distance per feature between the two datasets. The most-shifted features coincide with the model’s top SHAP drivers.

5.11 Error analysis

On the clean in-distribution test set of 47,074 URLs the detector makes very few errors: 15 false positives and 119 false negatives (Table 17). Table 18 lists representative examples and is more informative than the counts. The false positives are ordinary legitimate sites that happen to look structurally unusual — numeric domains such as *180360.com*, an uncommon TLD in *thefeedbackloop.xyz*, or the long multi-label host *provincialarchives.alberta.ca*. The false negatives are more instructive: *what.promerc.repl.co* is phishing hosted on a legitimate developer-sandbox domain — an in-the-wild instance of exactly the home-page-mimicry attack modelled in Section 5.7 — and *microsoft-sicherheitsupdate.com* is a brand-impersonation domain that the detector scores at 0.108, because it is short, HTTPS and structurally clean despite containing an obvious brand name. The rarity of in-distribution errors, contrasted with the severity of the mimicry and cross-dataset failures, demonstrates that clean-data error analysis alone would have entirely missed this report’s central finding.

item	count
false_positive	15
false_negative	119
test_size	47074

Table 17. Error counts on the clean in-distribution test set.

type	URL	y	p
FP	https://www.180360.com	0	0.876
FP	https://www.betterplants.basf.us	0	0.706
FP	https://www.10000birds.com	0	0.561
FP	https://www.thefeedbackloop.xyz	0	0.733
FP	https://www.provincialarchives.alberta.ca	0	0.746
FN	https://what.promerc.repl.co	1	0.011
FN	https://s.smbacmzu.icu	1	0.103
FN	https://www.microsoft-sicherheitsupdate.com	1	0.108
FN	https://www.nailcanhillrun.com.au	1	0.003
FN	https://www.2023foco.com.br	1	0.015

Table 18. Representative errors. The false negatives include phishing on legitimate hosting infrastructure and a structurally clean brand-impersonation domain.

6. Discussion

The four research questions can now be answered directly. **RQ1 (signal):** very little of the headline near-perfect score reflects transferable phishing-specific signal. One provided feature is label leakage, and once it is excluded the remaining score rests on a collection artifact that a one-line rule already captures at F1 0.68. **RQ2 (robustness):** the detector is genuinely robust to cheap character-level evasion, but this robustness is a side-effect of using structural features rather than a designed defence, and it offers no protection against an attacker who simply uses legitimate HTTPS hosting — an attack that reduces recall to zero. **RQ3 (generalisation):** the score does not transfer; on an independent corpus the model's ranking inverts, which is a more severe failure than the miscalibration usually assumed in domain-adaptation settings, and one that threshold tuning cannot repair. **RQ4 (explainability):** SHAP and distributional distance independently identify the same features as both the model's strongest drivers and the least transferable signals, which is what elevates the central claim from an observation to a causal explanation.

Taken together these results support a specific methodological conclusion: for URL-based phishing detection, in-distribution benchmark accuracy is close to uninformative about deployment behaviour. Evaluation practice should therefore report cross-dataset performance and adversarial recall as first-class results rather than as optional robustness checks. It is worth noting that the companion content-based detector, built on entirely different features and data, reached a structurally similar conclusion — a near-perfect in-distribution text classifier that degraded sharply out-of-distribution and under adversarial perturbation. That two independent signals exhibit the same fragility suggests the issue is not specific to URL features but is a general property of narrow, single-signal phishing classifiers evaluated only in-distribution.

7. Limitations and Future Work

Several limitations bound these conclusions. The cross-dataset comparison is informative but not perfectly controlled: PhiUSIIL and the Kaggle corpus differ in collection methodology as well as in genuine URL characteristics, so part of the measured shift may reflect crawling or encoding conventions rather than properties of phishing URLs themselves. The de-biased variant, which fails even more severely, is offered as partial evidence that the result is not attributable to a single confounding feature, but a fully controlled comparison would require corpora collected under a shared protocol. The feature set is intentionally restricted to string-derived signals and omits network-observable and third-party evidence — domain age from WHOIS, hosting and certificate reputation, and redirect-chain analysis — any of which could plausibly improve cross-dataset robustness, and precisely because they are harder for an attacker to control they are the most promising extension. The live-feed evaluation uses a single snapshot of 300 URLs from one provider and should be read as indicative rather than as a temporal-drift study. Finally, the adversarial evaluation covers four attack families chosen to span cheap and expensive evasion; it is not exhaustive.

One numerical caveat deserves explicit mention. Because the out-of-distribution scores are saturated — almost every URL in the independent corpus receives a predicted probability above 0.9 (Section 5.9) — the *ordering* within that band is decided by very small score differences, and the resulting AUC is correspondingly sensitive: differences at the level of floating-point rounding in the feature values shift the reported figure by a few hundredths (observed range approximately 0.39–0.43 across representations of the same features). The pipeline therefore always consumes the persisted feature artifact, so that a run reproduces identically regardless of whether the cache was already present. This sensitivity does not affect any conclusion drawn here: every variant is far below the 0.5 threshold that separates a correctly-ordered ranking from an inverted one, and the F1 figure (0.666) is stable across all of them. It is reported because a metric that moves with numerical noise should be flagged rather than quoted to three decimal places as though it were precise.

The clearest next step follows directly from Section 5.7: because no defence within the URL-only feature space closes the mimicry gap, the natural remedy is **fusion** with the companion content-based detector, so that an attacker who defeats one channel by using legitimate infrastructure must simultaneously defeat an independent second channel that reads entirely different evidence.

8. Conclusion

A URL-only phishing detector can be made fast, interpretable and robust to cheap character-level obfuscation, and it continues to catch a large share of current real-world phishing traffic. However, the near-perfect accuracy such detectors report on standard benchmarks is substantially a property of how those benchmarks were constructed rather than of phishing itself. It is supported in part by outright label leakage, it rests otherwise on a collection artifact that a single rule can capture, it is trivially evaded by an attacker willing to use ordinary legitimate infrastructure, and it does not survive contact with an independently collected dataset — where the model's ranking inverts entirely. We conclude that URL structure is a useful but insufficient signal on its own, and is best deployed as one independent component of a fused, multi-signal detection system.

References

- [1] Prasad, A. & Chandra, S. (2024). PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. *Computers & Security*, 136. Dataset: UCI Machine Learning Repository, id 967.
- [2] Siddhartha, M. (2021). *Malicious URLs Dataset* (651,191 URLs). Kaggle.
- [3] OpenPhish. *Community phishing feed*. <https://openphish.com> (snapshot retrieved 2026).
- [4] Ma, J., Saul, L. K., Savage, S., & Voelker, G. M. (2009). Beyond blacklists: learning to detect malicious web sites from suspicious URLs. *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1245–1254.
- [5] Le, H., Pham, Q., Sahoo, D., & Hoi, S. C. H. (2018). URLNet: learning a URL representation with deep learning for malicious URL detection. *arXiv preprint arXiv:1802.03162*.
- [6] Boucher, N., Shumailov, I., Anderson, R., & Papernot, N. (2022). Bad characters: imperceptible NLP attacks. *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004.
- [7] Lundberg, S. M. & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.